Complete finite program for the projective-line theory

```
# Exact finite applications for the PSL2/PGL2 classification.
# GAP's finitely presented group algorithms are retained computational machinery.
orb:=[]; iso:=fail; vertices:=[];
Fail := function(msg) Error(msg); end;
Need := function(b,msg) if not b then Fail(msg); fi; end;
IP := function(G, rr)
  local ss, hs, a,b;
  ss:=Combinations([1..Length(rr)]);
  hs:=List(ss,s->Subgroup(G,rr{s}));
  for a in [1..Length(ss)] do for b in [1..Length(ss)] do
    if Intersection(hs[a],hs[b])<>hs[Position(ss,Intersection(ss[a],ss[b]))] then
    return false; fi;
  od; od; return true;
end;
# The small isomorphisms used in the written applications.
for pp in [[PSL(2,2),SymmetricGroup(3)],[PSL(2,3),AlternatingGroup(4)],
           [PGL(2,3),SymmetricGroup(4)],[PGL(2,5),SymmetricGroup(5)]] do
  Need(IsomorphismGroups(pp[1],pp[2])<>fail,"small projective-group
  identification");
od;
# Every ordered irreducible string C-marking in the two exceptional groups.
for pair in [[SymmetricGroup(4),[[3,3,4],[3,4,3],[4,3,3]]],
             [AlternatingGroup(5),[[3,5,5],[5,3,5],[5,5,3]]]] do
  G:=pair[1]; inv:=Filtered(Elements(G),x->Order(x)=2); markings:=[];
  for a in inv do for c in inv do if a<>c and a*c=c*a then
    for b in inv do rr:=[a,b,c];
      if Order(a*b)>2 and Order(b*c)>2 and Subgroup(G,rr)=G and IP(G,rr) then
      Add(markings,rr); fi;
    od;
  fi; od; od;
  aut:=AutomorphismGroup(G); remaining:=ShallowCopy(markings); types:=[];
  while Length(remaining)>0 do
    rr:=remaining[1]; orb:=Orbit(aut,rr,OnTuples);
    Add(types,[Order(rr[1]*rr[2]),Order(rr[2]*rr[3]),Order(Product(rr))]);
    remaining:=Filtered(remaining,x->not x in orb);
  od;
  Sort(types); Need(types=pair[2],"exceptional rank-three marking census");
  F3:=FreeGroup("x","y","z"); z3:=GeneratorsOfGroup(F3);
  for typ in types do
    R3:=Concatenation(List(z3,a->a^2),[(z3[1]*z3[3])^2,
      (z3[1]*z3[2])^typ[1],(z3[2]*z3[3])^typ[2],Product(z3)^typ[3]]);
    Need(Size(F3/R3)=Size(G),"complete rank-three presentation");
  od;
  Print("RANK3 ",Size(G)," MARKINGS ",Length(markings)," TYPES ",types,"\n");
od;
# [facet m,n,k; vertex m,n,k; universal order; string IP; face counts]
rows := [
 [[5,5,3],[5,5,3],1,false,[]],
 [[5,5,3],[5,3,5],1,false,[]],
 [[5,3,5],[3,5,5],3420,true,[57,171,171,57]],
```

```
 [[5,3,5],[3,4,3],60,false,[]],
 [[5,3,5],[3,3,4],1,false,[]],
 [[4,3,3],[3,4,3],96,true,[4,6,6,4]],
 [[4,3,3],[3,3,4],24,false,[]],
 [[3,5,5],[5,3,5],660,true,[11,55,55,11]],
 [[3,4,3],[4,3,3],1,false,[]],
 [[3,3,4],[3,3,4],120,true,[5,10,10,5]] ];
localtypes:=[[3,3,4],[3,4,3],[4,3,3],[3,5,5],[5,3,5],[5,5,3]];
allpairs:=Filtered(Cartesian(localtypes,localtypes),p->p[1][2]=p[2][1]);
revtype:=t->[t[2],t[1],t[3]];
revpair:=p->[revtype(p[2]),revtype(p[1])];
covered:=Set(Concatenation(List(rows,r->[r[1],r[2]]),List(rows,r->revpair([r[1],r[2
]]))));
Need(Length(allpairs)=14 and Set(allpairs)=covered,"exhaustive compatible local
pairs");
Need(Length(Filtered(allpairs,p->p=revpair(p)))=6,"six self-dual local pairs");
F:=FreeGroup("r0","r1","r2","r3"); x:=GeneratorsOfGroup(F);
for n in [1..Length(rows)] do
  row:=rows[n]; f:=row[1]; v:=row[2]; Need(f[2]=v[1],"shared edge label");
  rel:=List(x,a->a^2);
  Append(rel,[(x[1]*x[2])^f[1],(x[2]*x[3])^f[2],(x[3]*x[4])^v[2],
              (x[1]*x[3])^2,(x[1]*x[4])^2,(x[2]*x[4])^2,
              (x[1]*x[2]*x[3])^f[3],(x[2]*x[3]*x[4])^v[3]]);
  U:=F/rel; Need(Size(U)=row[3],"fp universal order"); if row[3]=1 then
  G:=Group(()); rr:=[(),(),(),()]; else iso:=IsomorphismPermGroup(U);
  Need(iso<>fail,"fp permutation isomorphism"); G:=Image(iso);
  rr:=List(GeneratorsOfGroup(U),a->Image(iso,a)); fi;
  Need(Size(G)=row[3],Concatenation("universal order row ",String(n)));
  good:=ForAll(rr,a->Order(a)=2) and Length(Set(rr))=4 and IP(G,rr);
  Need(good=row[4],"rank-four intersection property");
  if good then
    faces:=List([1..4],i->Index(G,Subgroup(G,rr{Difference([1..4],[i])})));
    Need(faces=row[5],"face counts");
  else faces:=[]; fi;
  if n in [3,8,10] then
    if n=3 then target:=PSL(2,19); elif n=8 then target:=PSL(2,11); else
    target:=SymmetricGroup(5); fi;
    Need(IsomorphismGroups(G,target)<>fail,"universal group identification");
    dual:=GroupHomomorphismByImages(G,G,rr,Reversed(rr));
    Need(dual<>fail and IsBijective(dual),"self-duality");
  fi;
  if n in [4,7] then
    if n=4 then target:=AlternatingGroup(5); else target:=SymmetricGroup(4); fi;
    Need(IsomorphismGroups(G,target)<>fail,"collapsed group identification");
    Need(Subgroup(G,rr{[1,2,3]})=G or Subgroup(G,rr{[2,3,4]})=G,"collapsed full
    facet or vertex figure");
  fi;
  if n=6 then
    V:=FittingSubgroup(G); Need(Size(V)=16 and IsElementaryAbelian(V),"row-six
    elementary normal subgroup");
    Need(IsomorphismGroups(FactorGroup(G,V),SymmetricGroup(3))<>fail,"row-six
    quotient");
```

```
    Need(Length(ComplementClassesRepresentatives(G,V))>0,"row-six splitting");
  fi;
  if n=8 then
    vertices:=RightCosets(G,Subgroup(G,rr{[2,3,4]}));
    edges:=RightCosets(G,Subgroup(G,rr{[1,3,4]}));
    ends:=List(edges,e->Filtered([1..11],i->not
    IsEmpty(Intersection(Set(e),Set(vertices[i])))));
    Need(ForAll(ends,e->Length(e)=2) and Length(Set(ends))=55,"11-cell complete
    edge graph");
  fi;
  Print("ROW ",n," ORDER ",Size(G)," IP ",good," FACES ",faces,"\n");
od;
Print("PHASE230_FINITE_PASS\n");
QUIT;
```